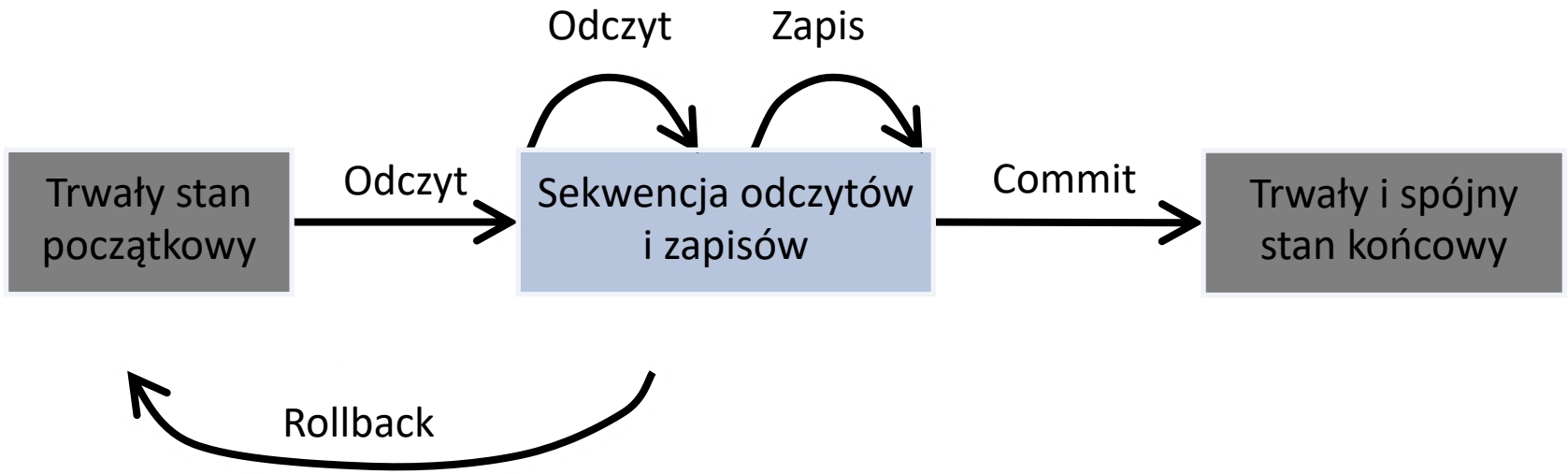
Zarządzanie danymi i wielodostęp



Transakcje

- Transakcja to sekwencja następujących po sobie akcji (operacji bazodanowych) traktowanych jak całość.
- Transakcja:
  - albo wykona się w całości,
  - nie wykona się wcale (żadna z jej akcji).
- Przykład transakcji: przelew bankowy: pobranie środków + przekazanie środków.



Transakcje przykłady

Wypłata 100 PLN

1. Odczytaj bieżące saldo
2. Nowe saldo = bieżące saldo – 100 PLN
3. Zapisz nowe saldo
4. Wydaj gotówkę

Przelew 100 PLN

1. Odczytaj bieżące saldo źródła
2. Nowe saldo źródła = bieżące – 100 PLN
3. Odczytaj bieżące saldo celu
4. Nowe saldo celu = bieżące + 100 PLN
5. Zapisz nowe saldo źródła
6. Zapisz nowe saldo celu

- Niektóre akcje mogą się nie powieść:
  - nastąpi uszkodzenie w obrębie aplikacji lub serwera bazodanowego,
  - wystąpi współistnienie innej transakcji

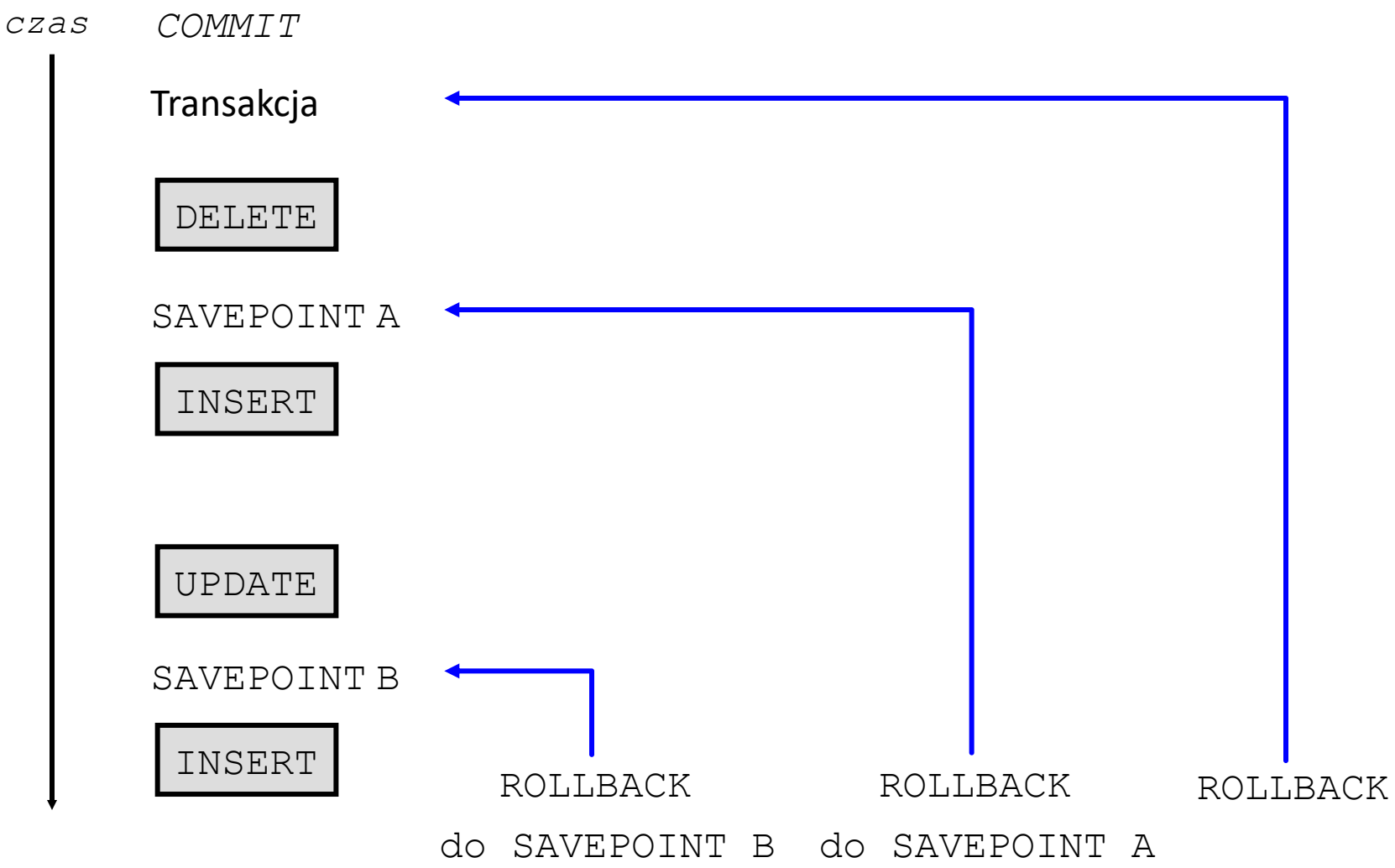
Transakcje – reguły ACID

- Atomowość (ang. Atomicity) – transakcja jest traktowana jako jednostka – zostanie wykonana w całości albo wcale
- Spójność (ang. Consistency) – transakcja zastaje bazę w spójnym stanie i w takim samym stanie ją pozostawia; wszystkie reguły bazodanowe muszą zostać uwzględnione (np. nie może być duplikatów dla kluczy podstawowych).
- Izolacja (ang. Isolation) – praca transakcji nie wpływa na inne, współbieżne transakcje; wyniki pracy transakcji mogą nie być widoczne dla innych transakcji przed jej zatwierdzeniem; widoczność jest określona przez poziom izolacji
- Trwałość (ang. Durability) – po zakończeniu transakcji jej rezultaty zapisywane są do bazy.

Transakcje w Oracle Database

- Transakcja bazodanowa może zawierać:
  - polecenia DML (jedno lub wiele)
  - jedno polecenie DDL
  - jedno polecenie DCL
- Transakcja rozpoczyna się , gdy wykonywane jest pierwsze polecenie.
- Transakcja kończy się, gdy:
  - wykonywane jest polecenie COMMIT lub ROLLBACK
  - wykonywane jest polecenie DDL lub DCL („autocommit”)
  - użytkownik kończy sesję SQL Developer lub SQL*Plus (COMMIT)
  - system ulega awarii (ROLLBACK)
- Ustawienie AUTOCOMMIT po każdym poleceniu DML:
  - w SQL*Plus: SET AUTOCOMMIT ON/OFF
  - w SQL Developer: Tools | Preferences | Database | Worksheet Parameters | “Autocommit in SQL Worksheet”

Transakcje bazodanowe – obsługa jawna



Mechanizm MVCC i status danych vs COMMIT i ROLLBACK

- Mechanizm MVCC (ang. Multiversion concurrency control) zapewnia współbieżny dostęp do bazy danych.
- Przed COMMIT/ROLLBACK :
  - Poprzedni stan danych może zostać odzyskany.
  - Bieżący użytkownik widzi stan danych po poleceniach DML.
  - Inni użytkownicy widzą stan danych przed poleceniami DML.
  - Modyfikowane wiersze są zablokowane – inni użytkownicy nie mogą ich modyfikować.
- Po COMMIT/ROLLBACK:
  - Zmiany danych są zapisywane w bazie.
  - Wszyscy użytkownicy widzą rezultaty.
  - Zwalniane są blokady na zasoby.
  - Usuwane są wszystkie punkty zachowania (savepoints).

Zatwierdzanie i wycofywanie zmian

```
DELETE FROM employees
WHERE  employee_id = 99999;


INSERT INTO departments
VALUES (290, 'Corporate Tax', NULL, 1700);
```

```
COMMIT;
```

```
DELETE FROM copy_emp;
ROLLBACK ;
```

Zatwierdzanie i wycofywanie zmian

```
DELETE FROM test;
25,000 rows deleted.

ROLLBACK;
Rollback complete.

DELETE FROM test WHERE  id = 100;
1 row deleted.

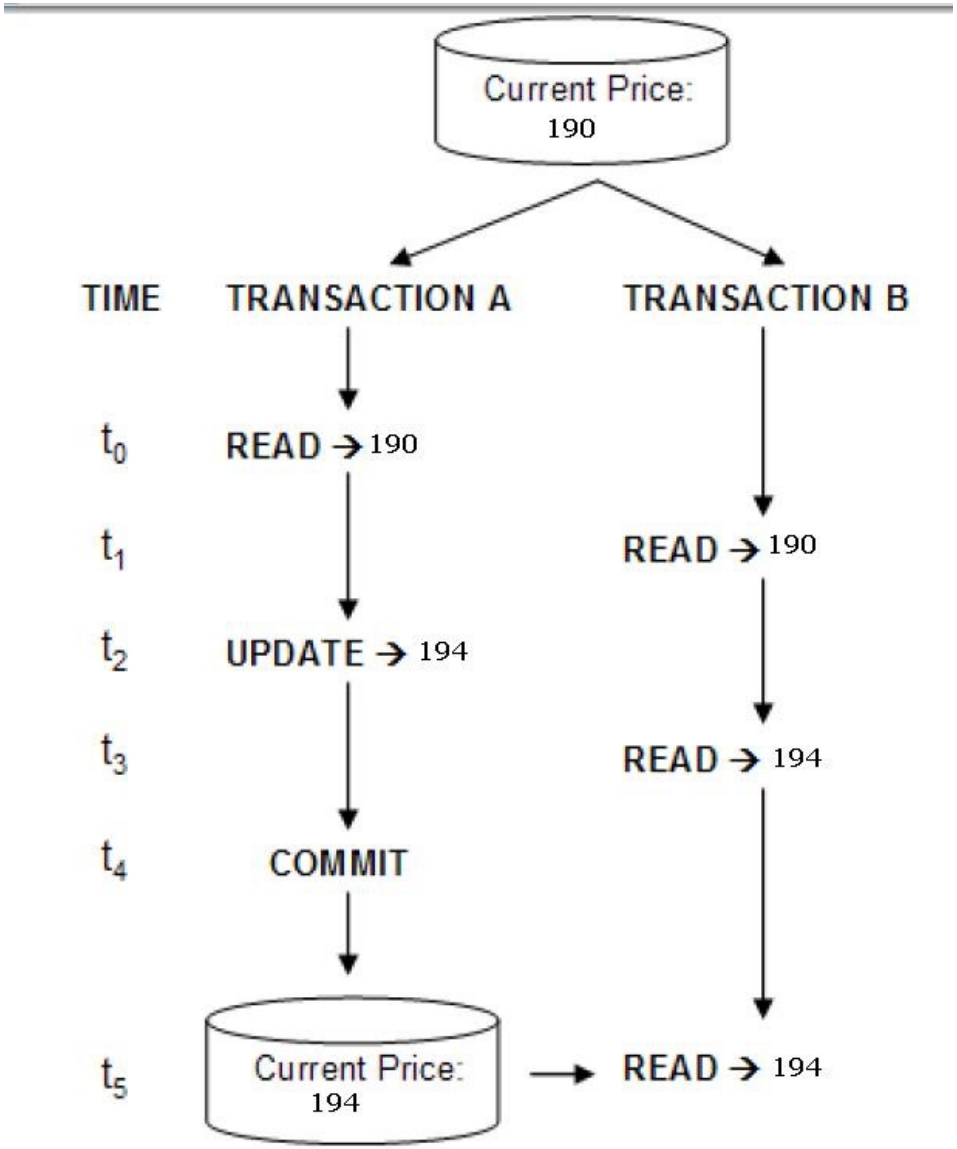
SELECT * FROM   test WHERE  id = 100;
No rows selected.

COMMIT;
Commit complete.
```

Możliwe sytuacje podczas transakcji współbieżnych

- **Brudne odczyty**

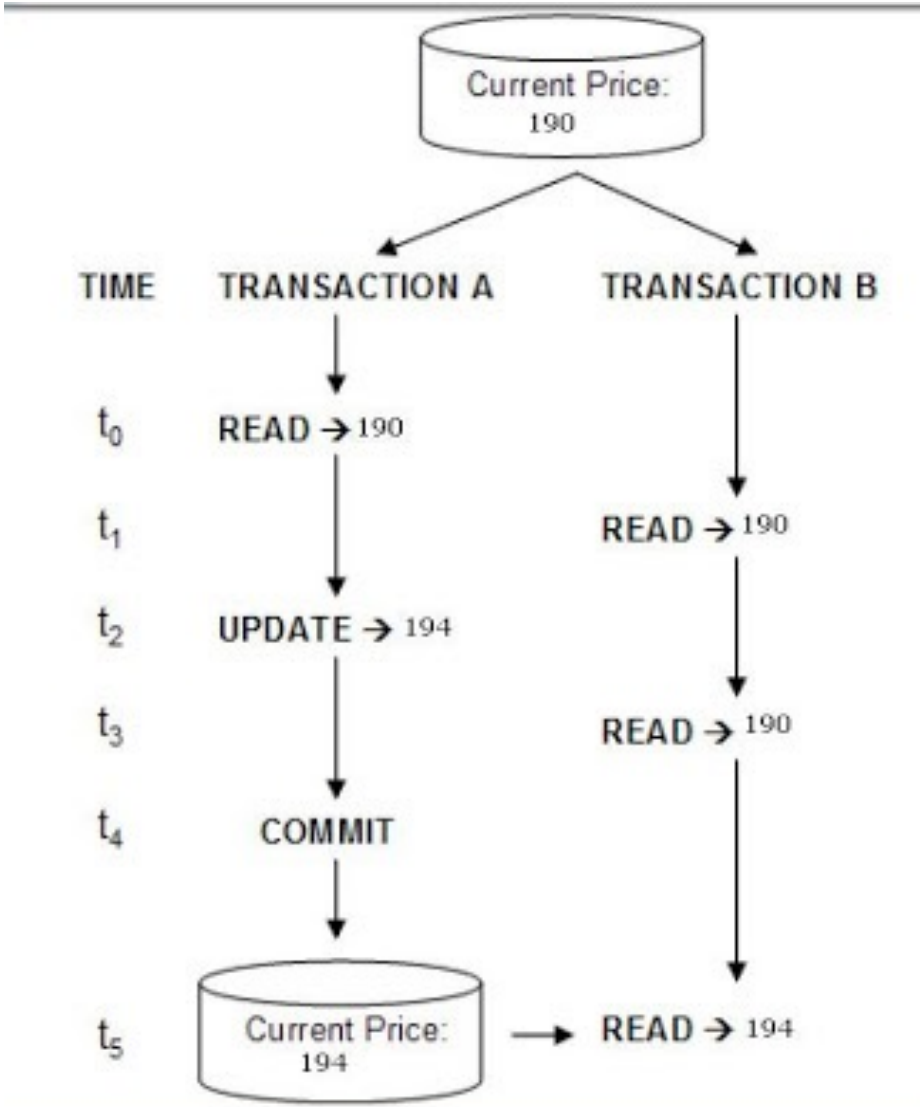
  (ang. *Dirty reads / uncommited dependency*) – transakcja odczyta dane zapisane przez inną transakcję, ale jeszcze nie potwierdzone (użytkownik może się pomylić / rozmyślić i jeszcze raz zmienić dane).



Możliwe sytuacje podczas transakcji współbieżnych

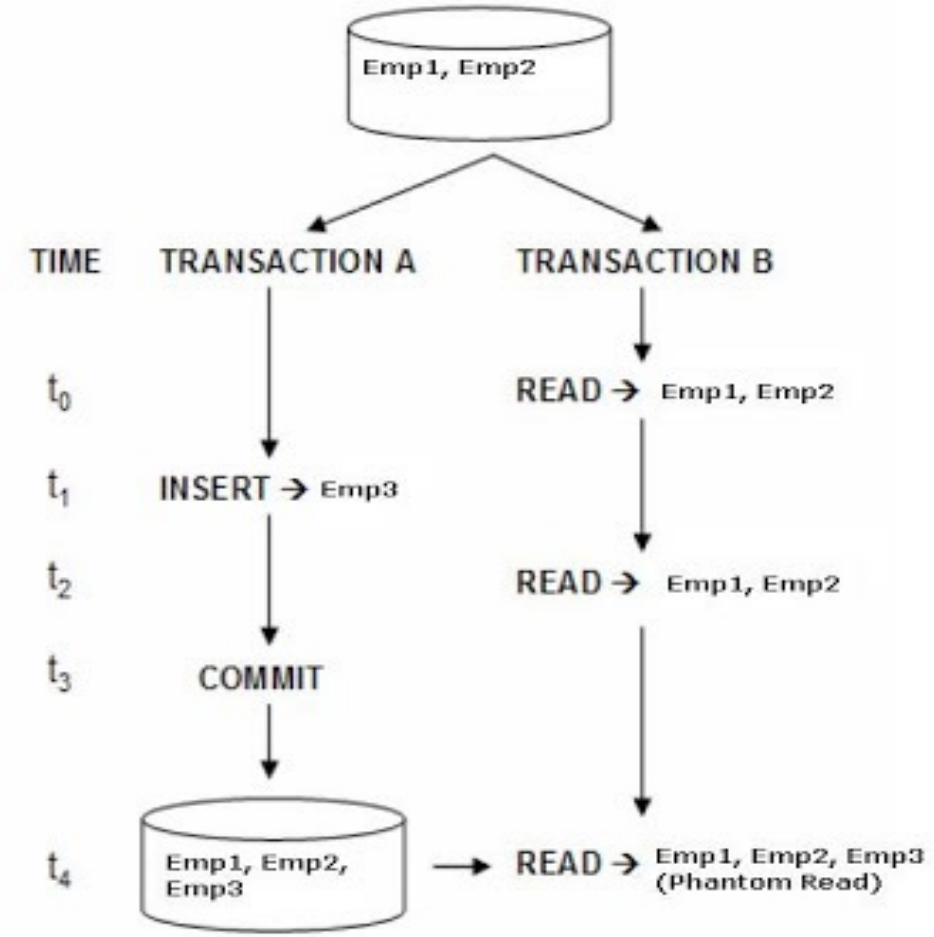
- **Niepowtarzalne odczyty**

  (ang. *Nonrepeatable (fuzzy) reads / inconsistent analysis*) – transakcja odczytując ponownie dane otrzymuje wyniki różniące się od poprzednio odczytanych (inna transakcja zmodyfikowała dane) - użytkownik odczytuje zatwierdzone dane, ale za każdym razem różne



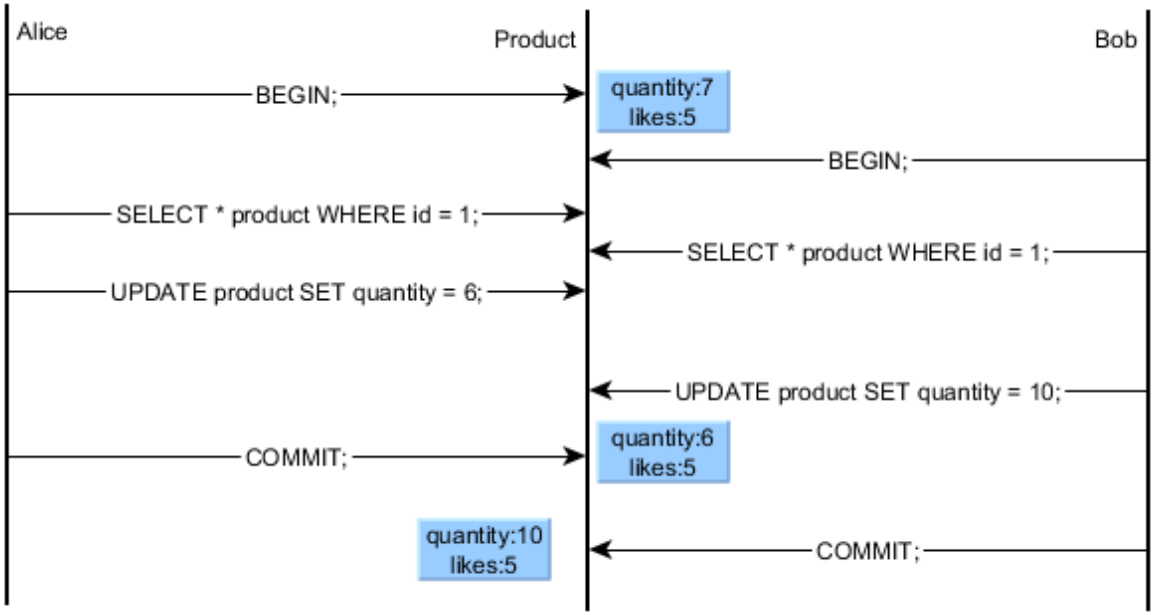
Możliwe sytuacje podczas transakcji współbieżnych

- **Odczyt widmo** (ang. *Phantom reads / phantoms*) – transakcja przy kolejnym odczycie zwraca zbiór danych spełniających predykat warunkowy, który różni się od poprzedniego (w wyniku dodania lub usunięcia wierszy) – pojawiają się lub znikają wiersze danych.



Możliwe sytuacje podczas transakcji współbieżnych

- **Utracone modyfikacje** (ang. *lost updates*) – dwie sesje modyfikują te same wiersze w tym samym czasie, przy czym w efekcie zapis uwzględni zmianę z jednej sesji („zwycięzca”) – druga z sesji nie będzie świadoma, że jej zmiana nie została uwzględniona („przegrany”).



- **Brakujące lub powielone odczyty** (ang. *missing reads & double reads*) – wiersze już odczytane zostały przeniesione do nowej jeszcze nieprzeskanowanej lokalizacji lub wiersze jeszcze nieodczytane zostały przeniesione do już przeskanowanej lokalizacji.

Modele współbieżności i poziomy izolacji

Kontrola współbieżności zakłada dwa modele:

Model pesymistyczny

- Odpowiedni system blokad uniemożliwia użytkownikom modyfikację danych w sposób wpływający na innych użytkowników.
- Po wykonaniu przez użytkownika akcji, która powoduje nałożenie blokady, inni użytkownicy nie mogą wykonywać akcji, które kolidowałyby z blokadą, dopóki właściciel jej nie zwolni.
- Używany w środowiskach, w których występuje duża rywalizacja związana z dostępem do danych i gdzie koszt zabezpieczania danych za pomocą blokad jest niższy niż koszt wycofywania transakcji w przypadku wystąpienia konfliktów współbieżności.

Modele współbieżności i poziomy izolacji

Kontrola współbieżności zakłada dwa modele:

Model optymistyczny

- Użytkownicy nie blokują danych podczas ich czytania.
- Gdy użytkownik aktualizuje dane, system sprawdza, czy inny użytkownik zmienił dane po tym, jak zostały odczytane. Jeśli inny użytkownik zaktualizował dane, zgłaszany jest błąd.
- Zazwyczaj użytkownik otrzymujący błąd wycofuje transakcję i zaczyna od nowa.
- Używany głównie w środowiskach, w których rywalizacja o dostęp do danych jest niewielka, a koszt okazjonalnego wycofania transakcji jest niższy niż koszt zablokowania danych podczas odczytu.

Poziomy izolacji

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read
Read uncommitted	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible
Serializable	Not possible	Not possible	Not possible

- Maksymalna izolacja jest równoznaczna z minimalną współbieżnością (i odwrotnie)
- Oracle Database oferuje poziomy izolacji: *read committed* (domyślnie), oraz *serializable*.
- Dodatkowy poziom izolacji *Read-Only*.

Isolation level \ Read phenomena	Dirty reads	Lost updates	Non-repeatable reads	Phantoms
Read Uncommitted	may occur	may occur	may occur	may occur
Read Committed	don't occur	may occur	may occur	may occur
Repeatable Read	don't occur	don't occur	don't occur	may occur
Serializable	don't occur	don't occur	don't occur	don't occur

Poziomy izolacji

	Dirty Reads	Lost Updates	Nonrepeatable reads	Phantom reads	Concurrency model
Read Uncommitted	Yes	Yes	Yes	Yes	Pessimistic
Read Committed	No	Yes	Yes	Yes	Pessimistic
Repeatable Read	No	No	No	Yes	Pessimistic
Serializable	No	No	No	No	Pessimistic
Snapshot	No	No	No	No	Optimistic
Read Committed Snapshot	No	Yes	Yes	Yes	Optimistic

From Locks to No Locks – Concurrency in SQL Server

Poziomy izolacji

`dirty read`

A transaction reads data written by a concurrent uncommitted transaction.

`nonrepeatable read`

A transaction re-reads data it has previously read and finds that data has been modified by another transaction (that committed since the initial read).

`phantom read`

A transaction re-executes a query returning a set of rows that satisfy a search condition and finds that the set of rows satisfying the condition has changed due to another recently-committed transaction.

`serialization anomaly`

The result of successfully committing a group of transactions is inconsistent with all possible orderings of running those transactions one at a time.

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

https://www.postgresql.org/docs/14/transaction-iso.html

Poziomy izolacji

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

https://www.postgresql.org/docs/14/transaction-iso.html

Poziomy izolacji – read committed

- Odpowiednie dla systemów, w których konflikty transakcji nie zdarzają się często.
- Każde wykonane zapytanie w obrębie transakcji widzi tylko te dane, które zostały zatwierdzone **przed zapytaniem** (nie całą transakcją).
- **ALE**: Zapytanie nie odczyta danych zatwierdzanych w czasie trwania tego zapytania, czyli: jeśli zapytanie przeskanowało połowę z tabeli o milionie wierszy i jeśli inna transakcja zmodyfikowała i zatwierdziła w tym czasie wiersz o numerze 950 000, to ta zmiana nie będzie widoczna.
- Modyfikacja danych odczytywanych przez inną transakcję jest możliwa i kolejny odczyt uwzględni zmiany – niepowtarzalne odczyty i odczyty fantomowe.

Poziomy izolacji – serializable

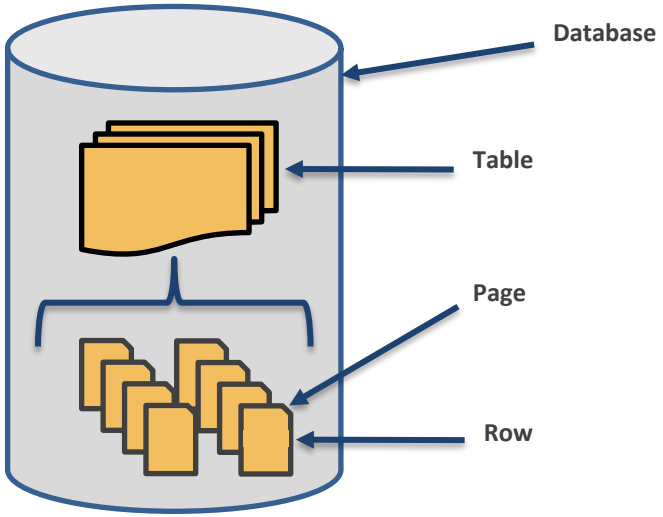
- Transakcja widzi tylko zmiany zatwierdzone przed rozpoczęciem transakcji oraz własne zmiany.
- Poziom izolacji serializable jest odpowiedni dla systemów:
  - z dużymi bazami danych oraz krótkimi transakcjami, które aktualizują tylko kilka wierszy.
  - w których prawdopodobieństwo, że dwie równoległe transakcje będą modyfikowały te same wiersze jest bardzo niskie.

Poziom izolacji Read-Only

- Zbliżony do poziomu serializable – dodatkowo nie pozwala na modyfikację danych z transakcji użytkownikom innym niż SYS, czyli zapobiega błędowi ORA-08177 - ORA-08177: can't serialize access for this transaction (stwierdzono dane zmodyfikowane przez operację, która pojawiła się po rozpoczęciu transakcji z poziomem serializable)
- Przydatny w sytuacji, gdy należy wykonać raport, który ma być spójny w momencie rozpoczęcia transakcji.
- Serwer Oracle uzyskuje spójność odczytu poprzez odczytywanie danych z segmentów wycofania (UNDO), jednak długie raporty mogą sięgnąć po zbyt wczesne dane (segmenty wycofania są zapełniane cyklicznie i inna transakcja może ich chcieć użyć) i w efekcie spowodować błąd „snapshot too old ” (można z kolei zapobiec poprzez odpowiednie ustawienia czasu retencji wycofania).

Blokady

- Zanim baza danych pozwali na zmodyfikowanie danych sesji, sesja najpierw musi zablokować dane, które ulegają modyfikacji.
- Blokada daje sesji wyłączną kontrolę nad danymi, tak aby żadna inna transakcja nie mogła modyfikować tych danych do czasu zwolnienia blokady.
- Transakcje mogą zablokować pojedyncze wiersze danych, kilka wierszy lub całe tabele.
- Istnieje wiele rodzajów blokad – zależnie od serwera baz danych.
- Blokady nałożone w ramach poszczególnych sesji wpływają na siebie (tzw. kompatybilność blokad).



- Jeśli blokady są niekompatybilne – sesja żądająca założenia niekompatybilnej blokady musi zaczekać, aż zostanie zwolniona konkurencyjna blokada.

Blokady

- Mechanizm blokowania ma na celu zapewnienie maksymalnego stopnia współbieżności danych w bazie.
- Transakcje, które modyfikują dane, najczęściej blokują na poziomie wiersza (nie na poziomie bloku lub tabeli).
- Oracle Database obsługuje ręczne i automatyczne blokowanie.
- Blokady zakładane automatycznie zawsze wybierają najniższy poziom blokowania, aby zminimalizować potencjalne konflikty z innymi transakcjami.

Transakcja 1



Transakcja 2



```
SQL> UPDATE employees
2   SET salary = salary + 100
3   WHERE employee_id = 100;
```

```
SQL> UPDATE employees
2   SET salary = salary * 1.1
3 WHERE employee_id = 100;
```

Mechanizm blokowania

- Zapytania o dane nie wymagają blokad i zapytanie zakończy się nawet, jeśli ktoś zablokował dane (zawsze pokazywane są dane sprzed blokady na podstawie informacji do wycofania).
- Jeśli wiele transakcji chce zablokować te same zasoby, blokadę założy pierwsza wnioskująca transakcja. Inne transakcje czekają na jej zakończenie. Mechanizm kolejkowania jest automatyczny i nie wymaga działań administracyjnych.
- Wszystkie blokady są uwalniane na końcu transakcji. W przypadku niepowodzenia transakcji, ten sam proces tła, który automatycznie wycofuje zmiany, zwalnia wszystkie blokady.

Transakcja 1



Transakcja 2



```
SQL> UPDATE employees
2   SET salary = salary + 100
3   WHERE employee_id = 100;
```

```
SQL> UPDATE employees
2   SET salary = salary*1.1
3   WHERE employee_id = 101;
```

Współbieżność danych

- Niski poziom blokowania sprawia, że różne transakcje mogą aktualizować poszczególne wiersze tej samej tabeli bez wzajemnej ingerencji.
- Chociaż model domyślny zakłada blokowanie na poziomie wiersza, Oracle umożliwia, w razie konieczności, ręczne blokowanie na wyższym poziomie:

```
SQL> LOCK TABLE employees IN EXCLUSIVE MODE;
Table(s) Locked.
```

Godzina:	Transakcja 1	`UPDATE hr.employees SET salary = salary + 100 WHERE employee_id = 100;`
	Transakcja 2	`UPDATE hr.employees SET salary = salary + 100 WHERE employee_id = 101;`
09:00:00	Transakcja 3	`UPDATE hr.employees SET salary = salary + 100 WHERE employee_id = 102;`
	…	…
	Transakcja *x*	`UPDATE hr.employees SET salary = salary + 100 WHERE employee_id = xxx;`

Rodzaje blokad

- **ROW SHARE** – pozwala na równoczesny dostęp do zablokowanej tabeli, ale zabrania sesjom blokowania całej tabeli na wyłączny dostęp;
- **ROW EXCLUSIVE** – analogicznie do ROW SHARE, ale zabrania także blokad w trybie SHARE; ROW EXCLUSIVE jest automatyczna podczas aktualizacji, wstawiania lub usuwania danych;
- **SHARE** – umożliwia jednoczesne zapytania, ale zabrania aktualizacji na zablokowanej tabeli; blokada SHARE jest wymagana (i automatyczna) podczas tworzenia indeksów na tabelach
- **SHARE ROW EXCLUSIVE** – używana do zapytań na całej tabeli; pozwala innym na zapytania do wierszy w tabeli; zabrania innym blokowania tabeli w trybie SHARE lub aktualizacji wierszy;
- **EXCLUSIVE** -pozwala na zapytania na zablokowanej tabeli, ale zabrania wszelkich innych działań; blokada EXCLUSIVE jest wymagana podczas usuwania tabeli (DROP);

Rodzaje blokad

- Ręczne blokady (tak jak inne) czekają na zwolnienie już istniejących blokad. Polecenie LOCK akceptuje specjalny argument kontrolujący oczekiwanie: NOWAIT.
- NOWAIT zwraca kontrolę natychmiast, jeśli dana tabela jest już zablokowana przez inną sesję (przerywa polecenie):

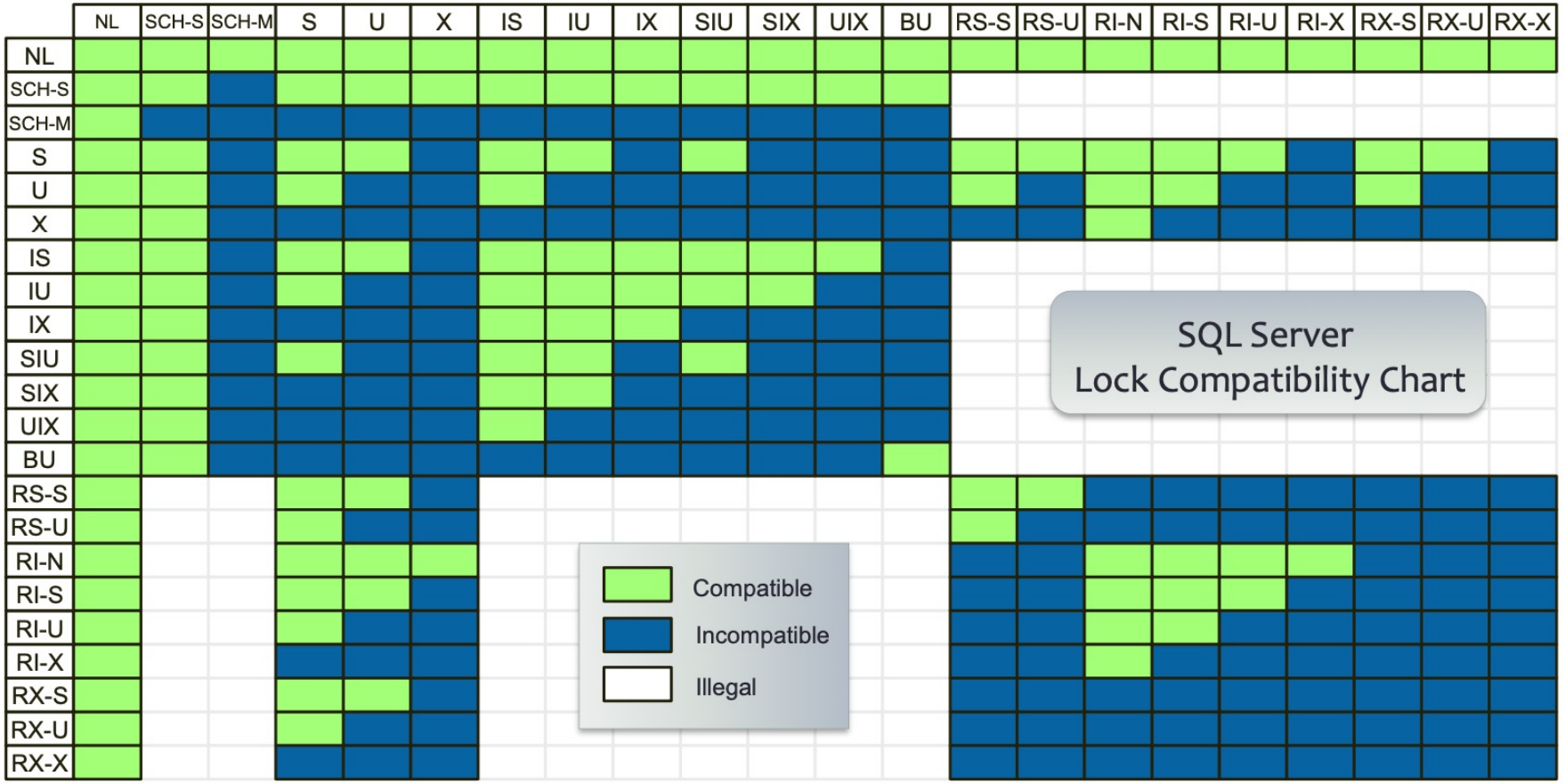
```
SQL> LOCK TABLE hr.employees IN SHARE MODE NOWAIT;
LOCK TABLE hr.employees IN SHARE MODE NOWAIT
*
ERROR at line 1:
ORA-00054: resource busy and acquire with NOWAIT
   specified
```

- Zazwyczaj nie jest konieczna ręczna blokada obiektów. Automatyczny mechanizm blokujący zapewnia współbieżny dostęp do potrzebnych danych dla większości zastosowań.

Rodzaje blokad

Lock mode	Description
Shared (S)	Used for read operations that do not change or update data, such as a SELECT statement.
Update (U)	Used on resources that can be updated. Prevents a common form of deadlock that occurs when multiple sessions are reading, locking, and potentially updating resources later.
Exclusive (X)	Used for data-modification operations, such as INSERT, UPDATE, or DELETE. Ensures that multiple updates cannot be made to the same resource at the same time.
Intent	Used to establish a lock hierarchy. The types of intent locks are: intent shared (IS), intent exclusive (IX), and shared with intent exclusive (SIX).
Schema	Used when an operation dependent on the schema of a table is executing. The types of schema locks are: schema modification (Sch-M) and schema stability (Sch-S).
Bulk Update (BU)	Used when bulk copying data into a table and the **TABLOCK** hint is specified.
Key-range	Protects the range of rows read by a query when using the serializable transaction isolation level. Ensures that other transactions cannot insert rows that would qualify for the queries of the serializable transaction if the queries were run again.

Rodzaje blokad – macierz kompatybilności



NL	No Lock	IX	Intent Exclusive	RI-S	Insert Range-Shared
SCH-S	Schema Stability Lock	SIU	Share with Intent Update	RI-U	Insert Range-Update
SCH-M	Schema Modification Lock	SIX	Share with Intent Exclusive	RI-X	Insert Range-Exclusive
S	Shared	UIX	Update with Intent Exclusive	RX-S	Exclusive Range-Shared
U	Update	BU	Bulk Update	RX-U	Exclusive Range-Update
X	Exclusive	RS-S	Shared Range-Shared	RX-X	Exclusive Range-Exclusive
IS	Intent Shared	RS-U	Shared Range-Update		
IU	Intent Update	RI-N	Insert Range-Null		

https://retracement.files.wordpress.com/2020/12/compatibility-matrix-1.23-bluegreen.pdf

Rodzaje blokad – macierz kompatybilności

	Existing granted mode					
Requested mode	**IS**	**S**	**U**	**IX**	**SIX**	**X**
Intent shared (IS)	Yes	Yes	Yes	Yes	Yes	No
Shared (S)	Yes	Yes	Yes	No	No	No
Update (U)	Yes	Yes	No	No	No	No
Intent exclusive (IX)	Yes	No	No	Yes	No	No
Shared with intent exclusive (SIX)	Yes	No	No	No	No	No
Exclusive (X)	No	No	No	No	No	No

From Locks to No Locks – Concurrency in SQL Server

Rodzaje blokad – macierz kompatybilności

- Blokady na poziomie tabeli:

	Existing Lock Mode							
Requested Lock Mode	ACCESS SHARE	ROW SHARE	ROW EXCL.	SHARE UPDATE EXCL.	SHARE	SHARE ROW EXCL.	EXCL.	ACCESS EXCL.
ACCESS SHARE								X
ROW SHARE							X	X
ROW EXCL.					X	X	X	X
SHARE UPDATE EXCL.				X	X	X	X	X
SHARE			X	X		X	X	X
SHARE ROW EXCL.			X	X	X	X	X	X
EXCL.		X	X	X	X	X	X	X
ACCESS EXCL.	X	X	X	X	X	X	X	X

- Blokady na poziomie wiersza:

	Current Lock Mode			
Requested Lock Mode	FOR KEY SHARE	FOR SHARE	FOR NO KEY UPDATE	FOR UPDATE
FOR KEY SHARE				X
FOR SHARE			X	X
FOR NO KEY UPDATE		X	X	X
FOR UPDATE	X	X	X	X

Blokady DML

Każda transakcja DML otrzymuje dwie blokady:

- blokadę EXCLUSIVE na aktualizowane wiersze;
- blokadę na poziomie tabeli ROW EXCLUSIVE, która ma zapobiec zablokowaniu całej tabeli przez inną sesję (prawdopodobnie z powodu usuwania lub obcinania tabeli);

Transakcja 1

```
SQL> UPDATE employees
2   SET salary=salary*1.1
3   WHERE employee_id= 107;
1 row updated.
```

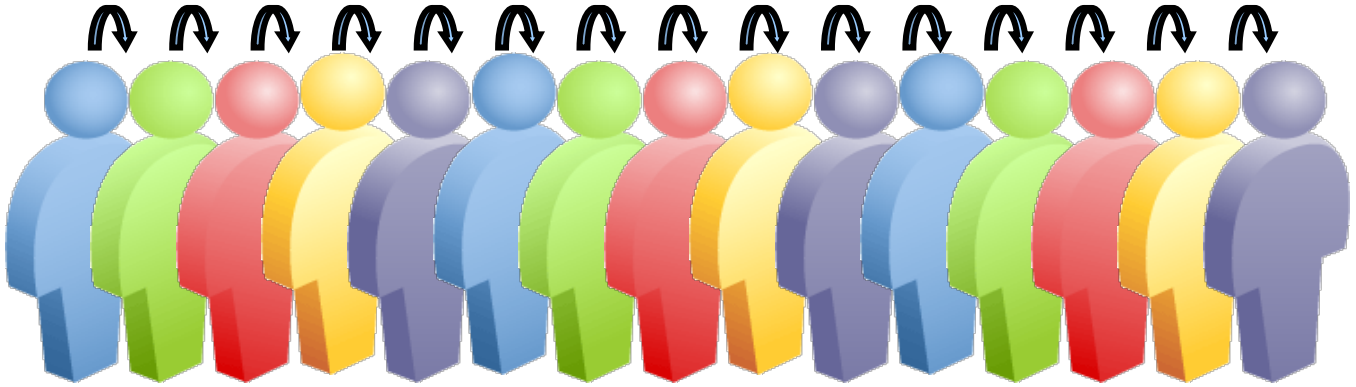
Transakcja 2

```
SQL> UPDATE employees
2   SET salary=salary*1.1
3   WHERE employee_id= 106;
1 row updated.
```

Mechanizm kolejkowania blokad

- Żądania blokad są automatycznie umieszczane w kolejce. Gdy tylko zakończy się transakcja trzymająca blokadę, otrzymuje ją kolejna sesja.
- Sesje, które już posiadają blokady mogą żądać zmiany blokady bez konieczności przechodzenia na koniec kolejki.

  Przykład: Sesja posiada blokadę SHARE na tabeli. Może zażądać przekształcenia blokady na tryb EXCLUSIVE. O ile nikt inny nie ma już założonej na tej tabeli blokady EXCLUSIVE lub SHARE, sesja otrzyma blokadę EXCLUSIVE bez czekania w kolejce.



Widoki słownika związane z blokadami

- V$TRANSACTION – dane o każdej aktywnej transakcji.
- V$SESSION – dane o zalogowanych sesjach.
- V$LOCK – dane o każdej nałożonej blokadzie, jak również o sesjach, które czekają na blokadę.

```
SQL> select a.SID "Blocking Session", b.SID "Blocked Session"
2  from v$lock a, v$lock b
3  where a.SID != b.SID
4  and a.ID1 = b.ID1
5  and a.ID2 = b.ID2
6  and b.request > 0
7  and a.block = 1;

Blocking Session Blocked Session
---------- --------------
49                 35
```

Konflikty blokad

- Konflikty blokad pojawiają się często, jednak są zwykle rozwiązywane poprzez upływ czasu oraz mechanizm kolejkowania. Czasami konflikt blokad może wymagać interwencji administratora.
- Przykład: Transakcja 2 tworzy blokadę dla jednego wiersza o 9:00:00 i zapomina zatwierdzić, pozostawiając blokadę. Transakcja 1 próbuje zaktualizować całą tabelę, co wymaga blokady wszystkich wiersze o 9:00:05. Transakcja 1 jest zablokowana przez transakcję 2 aż do 16:30:01, kiedy następuje COMMIT. Użytkownik próbujący wykonać transakcję 1 prawie na pewno skontaktuje się z administratorem w celu uzyskania pomocy. DBA musi wykryć i rozwiązać konflikt.

Transakcja 1	Czas	Transakcja 2
`UPDATE employees SET salary = salary +100 WHERE employee_id=100;` `1 row updated.`	9:00:00	`UPDATE employees SET salary =salary+100 WHERE employee_id=101;` `1 row updated.`

`UPDATE employees SET COMMISION_PCT =2 WHERE employee_id=101;`
Sesja czeka w kolejce (konflikt blokad).
	9:00:05 	`SELECT sum(salary) FROM employees;`
		`SUM(SALARY)`
		`-----------`
		`692634`

Sesja wciąż czeka!	16:30:00	**Różnego rodzaju operacje SELECT, INSERT, UPDATE i DELETE w ciągu ostatnich 7,5 godzin, bez zatwierdzenia lub wycofania!**
`1 row updated.` **Kontynuacja sesji.**	16:30:01	`commit;`

Możliwe przyczyny konfliktu blokad

- najczęściej – niezatwierdzone zmiany;
- długotrwałe transakcje: wiele aplikacji używa przetwarzania wsadowego, aby wykonać masowych aktualizacji; zadania te są zazwyczaj zaplanowane na godziny niskiej aktywności użytkownika lub jej braku; w niektórych przypadkach mogą się w zadanym czasie nie skończyć;
- niepotrzebne wysokie poziomy blokad: nie wszystkie bazy wspierają blokady niskiego poziomu (Oracle dodał wsparcie blokad na poziomie wiersza w 1988); niektóre bazy blokują na poziomie stron lub tabel (programiści aplikacji przeznaczonych dla różnych baz, często piszą swoje aplikacje ze sztucznie wysokim poziomem blokady tak, żeby baza danych Oracle działa podobnie do innych systemów, analogicznie programiści przechodzący z innych systemów do Oracle'a);



Rozwiązywanie konfliktów blokad

- Manipulacji sesjami można dokonywać poprzez wydawanie poleceń SQL.
- Szczegółowe informacje o wszystkich podłączonych sesjach zawiera tabela v$session. Identyfikator (ID) sesji blokującej zawiera kolumna blocking_session.
- Do zabicia sesji należy znać SID oraz SERIAL#.

1

```
SQL> select sid, serial#, username
                from v$session where sid in
                (select blocking_session from v$session)
```

Result:

SID	SERIAL#	USERNAME
144	8982	HR

2

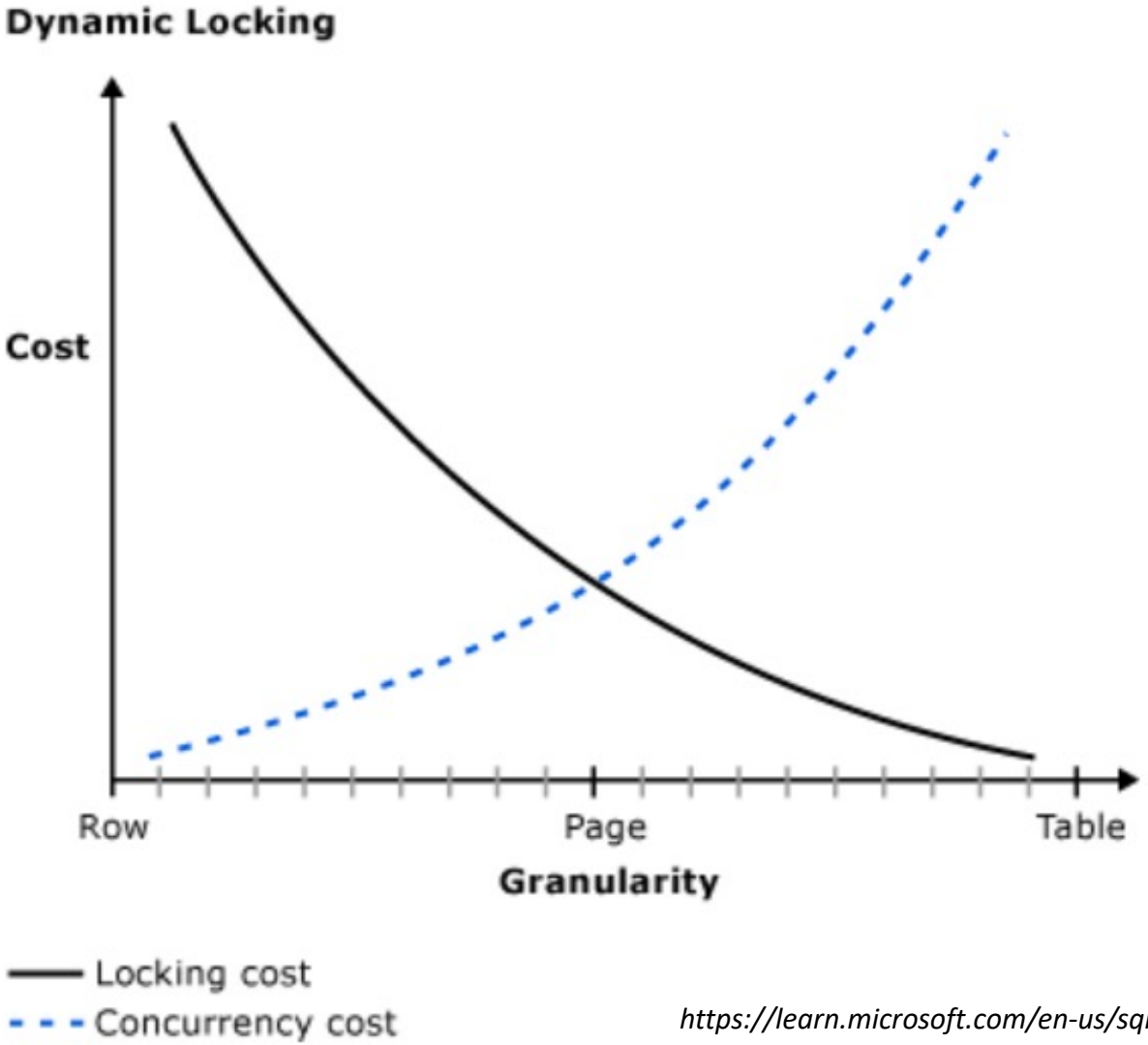
```
SQL> alter system kill session '144,8982' immediate;
```

Rozwiązywanie konfliktów blokad

- Rozwiązanie konfliktu blokad następuje w wyniku zwolnienia blokady przez trzymającą ją sesję. Najlepszym rozwiązaniem na zwolnienie blokady przez sesję jest skontaktowanie się z jej użytkownikiem i poproszenie o zakończenie transakcji.
- W nagłych przypadkach, administrator może zakończyć sesję przytrzymującą blokadę poprzez jej zabicie (opcja Kill Session). Wiąże się to z utratą (wycofaniem) wszystkich prac w ramach ostatniej transakcji. Użytkownik, którego sesja została zabita, musi zalogować się ponownie i powtórzyć wszystkie czynności od ostatniego zatwierdzenia.
- Użytkownicy, których sesje zostały zabite, otrzymują podczas kolejnego wydania polecenia SQL następujący komunikat o błędzie:

```
ORA-03135: connection lost contact
```

Koszt współbieżność vs blokady



https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver16

Zakleszczenia

- Zakleszczenie jest specjalnym przykładem konfliktu blokad – nie należy mylić ze zwykłą blokadą!
- Zakleszczenia powstają, gdy dwie lub więcej sesji czeka na dane wzajemnie zablokowane. Ponieważ każdy czeka na każdego, nikt nie może zakończyć transakcji w celu rozwiązania konfliktu.
- Oracle Database automatycznie wykrywa zakleszczenia i kończy błędem. Prawidłowa odpowiedź na ten błąd to zatwierdzenie lub wycofanie, które zwalniają blokady tak, aby inne sesje mogły kontynuować transakcje.
- Przykład: W odpowiedzi na wykryty błąd zakleszczenia, transakcja 1 musi zostać albo zatwierdzona albo wycofana. Jeśli zostanie zatwierdzona, trzeba będzie ponownie potwierdzić drugą aktualizację, aby zakończyć transakcję. Jeśli zostanie wycofana, trzeba będzie ponownie potwierdzić oba polecenia, aby zakończyć transakcję.

Transakcja 1		Transakcja 2

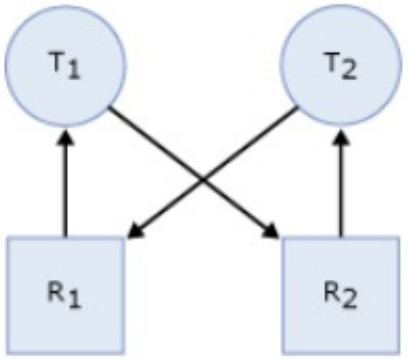
`UPDATE employees`
`SET salary = salary x 1.1`
`WHERE employee_id = 1000;`
	9:00	`UPDATE employees`
		`SET manager = 1342`
		`WHERE employee_id = 2000;`

`UPDATE employees`
`SET salary = salary x 1.1`
`WHERE employee_id = 2000;`
	9:15	`UPDATE employees`
		`SET manager = 1342`
		`WHERE employee_id = 1000;`

`ORA-00060:` `Deadlock detected while waiting for resource`	**9:16**	

Zakleszczenia

- W MS SQL Server wykrywanie zakleszczenia jest wykonywane przez wątek monitora blokady, który okresowo inicjuje przeszukiwanie wszystkich zadań.
- SQL Server wybiera jedną z sesji jako „ofiarę zakleszczenia” i bieżąca transakcja tej sesji jest przerywana z kodem błędu w celu usunięcia zakleszczenia.
- Domyślny interwał to 5 sekund.
- Jeśli wątek monitora blokad znajdzie zakleszczenie, interwał wykrywania zakleszczeń spadnie z 5 sekund do nawet 100 milisekund, w zależności od częstotliwości zakleszczeń. Jeśli wątek monitora blokady przestanie znajdować zakleszczenia, aparat bazy danych programu SQL Server zwiększa interwały między wyszukiwaniami do 5 sekund.



https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver16